

A parametric time-marching PINN for the FitzHugh–Nagumo benchmark of Ramsay et al. (2007)

Implementation details of
`ramsay_fhn_forward_parametric_PINNs-torch.ipynb`

September 28, 2026

Abstract

`ramsay_fhn_forward_parametric_PINNs-torch.ipynb` (folder `Nagumo-1D`) trains a physics-informed neural network (PINN) that solves the forward problem for the FitzHugh–Nagumo system of Ramsay, Hooker, Campbell & Cao (2007), Section 3.1, for *every* value of the parameter $a \in [0.1, 0.5]$ at once. The parameters b and c are fixed. There is no data fitting and no parameter estimation: the networks see only the ODE residual, and an RK45 solution is used purely for validation. The method has four ingredients:

- time-marching over ten windows of length 2, with one small MLP per window that takes (t, a) as input;
- a hard-constrained ansatz whose start state is the previous window’s end state, as a function of a ;
- a residual loss, scaled so both equations are $O(1)$, on a tensor grid of t and a values;
- Adam followed by L-BFGS, window by window.

In the example run (Section 8) the relative L^2 error against RK45 over 81 values of a has mean 1.4×10^{-3} and maximum 3.5×10^{-3} , after 161 s of training on one GPU.

Contents

1	Problem statement	2
2	Time-marching decomposition	2
3	Ansatz	3
3.1	Normalised inputs	3
3.2	Hard-constrained, parametric start state	3
3.3	Network	3
4	Residual and loss	4
4.1	Scaled residuals	4
4.2	Collocation on a tensor grid	4
4.3	Loss	4
5	Optimisation	4
6	Prediction and validation	5
7	Hyperparameters	5

1 Problem statement

The model, in the parameterisation of Ramsay et al. (2007), is

$$\frac{dV}{dt} = c \left(V - \frac{V^3}{3} + R \right), \quad (1)$$

$$\frac{dR}{dt} = -\frac{1}{c} (V - a + b R), \quad (2)$$

$$V(0) = V_0 = -1, \quad R(0) = R_0 = 1, \quad (3)$$

on $t \in [0, 20]$, with $b = 0.2$ and $c = 3$ fixed. The paper uses $a = 0.2$. Here a is a *parameter of the solution*: we seek

$$(V, R) : [0, 20] \times [a_{\min}, a_{\max}] \rightarrow \mathbb{R}^2, \quad (t, a) \mapsto (V(t; a), R(t; a)),$$

with $[a_{\min}, a_{\max}] = [0.1, 0.5]$, such that $(V(\cdot; a), R(\cdot; a))$ solves (1)–(3) for each a . After training, the solution for any a in the range is one forward pass through the networks, with no retraining.

Qualitative behaviour. The solution is a relaxation oscillation with period ≈ 9 . V has sharp upstrokes and downstrokes, with $|dV/dt|$ up to ≈ 5 , and R varies slowly. Table 1 shows that the timing of these fronts depends on a . The first upstroke hardly moves, but the second moves by 0.8 time units and the downstrokes by up to 2.5. At $a = 0.5$ the third upstroke falls outside $[0, 20]$. At a fixed t , V can differ by up to 3.3 between $a = 0.1$ and $a = 0.5$. The networks therefore have to represent sharp fronts whose position depends on a .

a	upstrokes			downstrokes	
0.1	0.58	9.53	18.40	5.36	14.24
0.2	0.57	9.61	18.58	5.67	14.65
0.3	0.56	9.76	18.90	6.04	15.18
0.4	0.56	9.99	19.39	6.47	15.87
0.5	0.55	10.34	–	7.00	16.76

Table 1: Times at which V crosses zero upwards (upstrokes) and downwards (downstrokes), from RK45 ($\text{rtol} = 10^{-10}$). The fronts move later as a increases.

2 Time-marching decomposition

The interval is split into $K = 10$ windows $[t_k, t_{k+1}]$, $t_k = 2k$, $k = 0, \dots, K - 1$. Each window has its own network $\mathcal{N}_{\theta_k}$, and the windows are trained one after the other. A window of length 2 is less than a quarter of a period, so each network only has to fit a short piece of the trajectory: at most one front, for every a .

This design is inherited from the fixed- a notebook `ramsay_fhn_forward_PINNs-torch.ipynb`. This note does not test a single network on $[0, 20] \times [0.1, 0.5]$ for comparison.

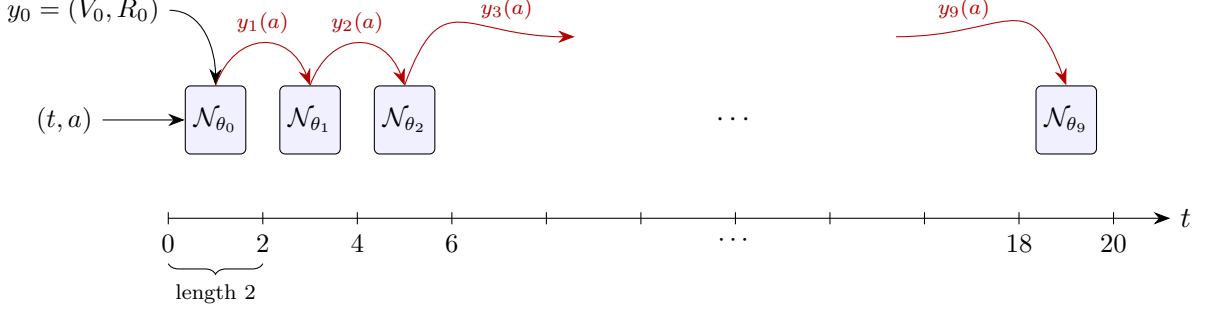


Figure 1: Time-marching. Window k has its own MLP with inputs (t, a) . Its start state $y_k(a)$ is the previous window’s output at (t_k, a) , evaluated for the same a (red arrows), and is frozen while window k is trained.

3 Ansatz

3.1 Normalised inputs

Inside window k , the network sees

$$\hat{t} = 2s - 1, \quad s = \frac{t - t_k}{t_{k+1} - t_k} \in [0, 1], \quad \hat{a} = 2 \frac{a - a_{\min}}{a_{\max} - a_{\min}} - 1 \in [-1, 1],$$

so both inputs lie in $[-1, 1]$ whatever the window and the range of a .

3.2 Hard-constrained, parametric start state

The approximation in window k is

$$u_k(t, a) = (V, R)(t, a) = y_k(a) + s \mathcal{N}_{\theta_k}(\hat{t}, \hat{a}), \quad (4)$$

where the start state $y_k(a) \in \mathbb{R}^2$ is defined recursively by

$$y_0(a) = (V_0, R_0), \quad y_{k+1}(a) = u_k(t_{k+1}, a). \quad (5)$$

The start state is a function of a : the solution at $t = t_k$ is different for each a .

Proposition 1. *For every $a \in [a_{\min}, a_{\max}]$ and all network weights, the piecewise function $u(t, a) = u_k(t, a)$ for $t \in [t_k, t_{k+1}]$ satisfies the initial condition (3) exactly and is continuous in t at every window boundary.*

Proof. At $t = t_k$ we have $s = 0$, so $u_k(t_k, a) = y_k(a)$. For $k = 0$ this is (V_0, R_0) . For $k \geq 1$ it equals $u_{k-1}(t_k, a)$ by (5), so the two pieces agree at t_k . $\square$

Only the ODE residual is left for the loss: there is no penalty term for the initial condition or for continuity, and so no weight to tune between them.

3.3 Network

Each $\mathcal{N}_{\theta_k} : \mathbb{R}^2 \rightarrow \mathbb{R}^2$ is an MLP with 3 hidden layers of width 32 and tanh activations. It has 2,274 weights, and all windows have the same architecture. Weights use Xavier-normal initialisation and biases are initialised to zero (class `WindowPINN`).

4 Residual and loss

4.1 Scaled residuals

In the paper’s form the V -equation is $O(c)$ and dR/dt is $O(1/c)$. The residuals are therefore scaled so that both are $O(1)$:

$$r_V(t, a) = \frac{1}{c} \partial_t V - \left(V - \frac{V^3}{3} + R \right), \quad (6)$$

$$r_R(t, a) = c \partial_t R + (V - a + b R). \quad (7)$$

Dividing (1) by c and multiplying (2) by c does not change the ODE or its solution, only the relative weight of the two equations in the loss. Here a enters twice: as a network input, and explicitly in r_R . The derivative ∂_t is computed by automatic differentiation with respect to t only (`torch.autograd.grad`). Derivatives with respect to a are never needed.

4.2 Collocation on a tensor grid

Window k uses the tensor grid

$$\mathcal{C}_k = \{t_k + (t_{k+1} - t_k) \frac{i}{N_t - 1}\}_{i=0}^{N_t-1} \times \{a_{\min} + (a_{\max} - a_{\min}) \frac{j}{N_a - 1}\}_{j=0}^{N_a-1},$$

with $N_t = 400$ and $N_a = 21$, i.e. $a = 0.10, 0.12, \dots, 0.50$ and 8,400 points per window. The grid is fixed, which L-BFGS needs, since it assumes the loss does not change between iterations. The start states $y_k(a_j)$ at the N_a training values of a are computed once, by marching through the windows already trained (`window_start_states`), and are held fixed (no gradient) while window k is trained.

4.3 Loss

The loss of window k is the mean squared residual over $\mathcal{C}_k$,

$$\mathcal{L}_k(\theta_k) = \frac{1}{N_t N_a} \sum_{(t_i, a_j) \in \mathcal{C}_k} r_V(t_i, a_j)^2 + r_R(t_i, a_j)^2. \quad (8)$$

Every training value of a has the same weight, so a single network has to fit all of them at once.

5 Optimisation

The windows are trained in order $k = 0, 1, \dots, K - 1$. Window k starts from a freshly initialised network. The windows before it are never updated again, so errors made in window k pass on to later windows through $y_{k+1}(a)$. Each window is trained in two stages:

1. **Adam**: 500 steps with learning rate 10^{-3} , to move the network away from its random initialisation;
2. **L-BFGS**: up to 3,000 iterations with history size 100, a strong-Wolfe line search, `tolerance_grad` = 10^{-12} and `tolerance_change` = 10^{-16} .

The whole batch of 8,400 points is used at every step. The loss history counts every Adam step and every L-BFGS function evaluation, including line-search evaluations.

Precision. The notebook keeps the device logic of the torch FitzHugh–Nagumo notebook: float32 on a GPU if one is available, otherwise float64 on the CPU. In float32, L-BFGS stops making progress at losses around 10^{-6} (Figure 2), which limits the accuracy that can be reached.

6 Prediction and validation

Prediction. `pinn_predict(t, a_values)` evaluates the trained PINN on any grid of t and a values. It marches through the windows: for each window k it evaluates (4) at the requested times in $[t_k, t_{k+1}]$ for all a at once, and then computes $y_{k+1}(a) = u_k(t_{k+1}, a)$ for the next window. The cost is one forward pass per window.

Reference and error metric. For each test value a , the reference solution comes from `scipy.integrate.solve_ivp` with RK45, $\text{rtol} = 10^{-10}$ and $\text{atol} = 10^{-12}$, on $n_{\text{eval}} = 4000$ equispaced times in $[0, 20]$. The reported error is

$$e(a) = \left(\frac{\sum_i (V_i - V_i^{\text{ref}})^2 + (R_i - R_i^{\text{ref}})^2}{\sum_i (V_i^{\text{ref}})^2 + (R_i^{\text{ref}})^2} \right)^{1/2},$$

together with the separate relative errors for V and R . It is evaluated on 81 equispaced values of a in $[0.1, 0.5]$. Only 21 of these are training values; the rest lie between them and so test how well the networks interpolate in a .

7 Hyperparameters

group	quantity	value
model	b, c (fixed)	0.2, 3
	a range	$[0.1, 0.5]$
	V_0, R_0	-1, 1
	time interval	$[0, 20]$
windows	length, number K	2, 10
collocation	N_t per window, N_a	400, 21
network	hidden layers $\times$ width	3×32 , tanh
	weights per window	2,274
Adam	steps, learning rate	500, 10^{-3}
L-BFGS	max. iterations, history	3000, 100
	line search	strong Wolfe
	tolerance (grad, change)	10^{-12} , 10^{-16}
validation	n_{eval} , test values of a	4000, 81
other	seed	0

Table 2: Settings of `ramsay_fhn_forward_parametric_PINNs-torch.ipynb` (parameter cell).

8 Example output

The figures below come from one run of the notebook as it stands (float32, one NVIDIA RTX A6000, seed 0). Training all ten windows took 161 s.

Training. Every window ends with a loss between 4.6×10^{-7} and 1.6×10^{-6} (Figure 2), after 1,079 to 3,241 loss evaluations. The windows in which a front’s position changes with a ([6, 12] and [14, 20], Table 1) need the most evaluations (2,246 to 3,241). Window $[0, 2]$ also contains a front, the first upstroke, but its position hardly depends on a , and it needs the fewest.

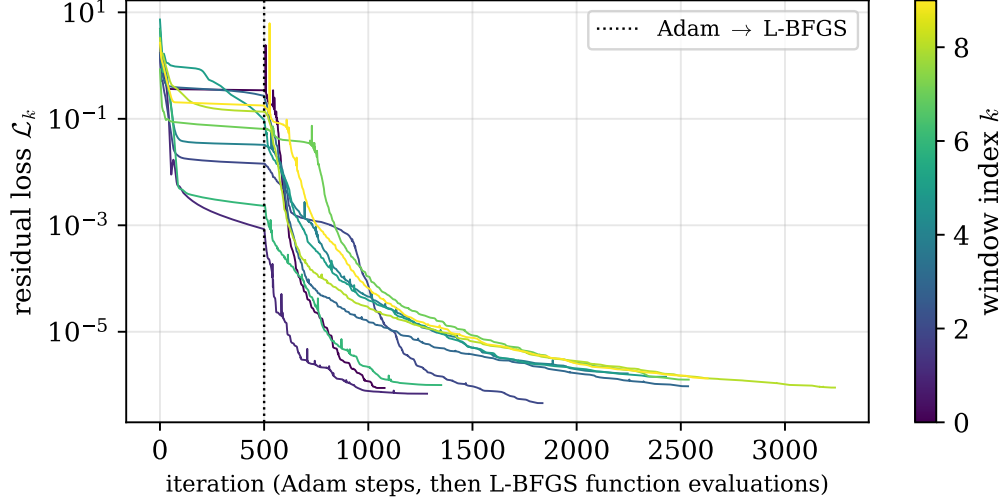


Figure 2: Loss $\mathcal{L}_k$ during training, one curve per window. The first 500 iterations are Adam steps; after that, each point is one L-BFGS function evaluation.

Error as a function of a . Over the 81 test values the error $e(a)$ has mean 1.4×10^{-3} , minimum 3.9×10^{-4} (at $a = 0.365$) and maximum 3.5×10^{-3} (at $a = 0.5$). The largest pointwise errors are 2.8×10^{-2} for V , at a front, and 4.3×10^{-3} for R . The error curve (Figure 3) varies smoothly and does not rise between training values of a : the error at values between them is similar to the error at the training values themselves. The error is largest at the two ends of the range.

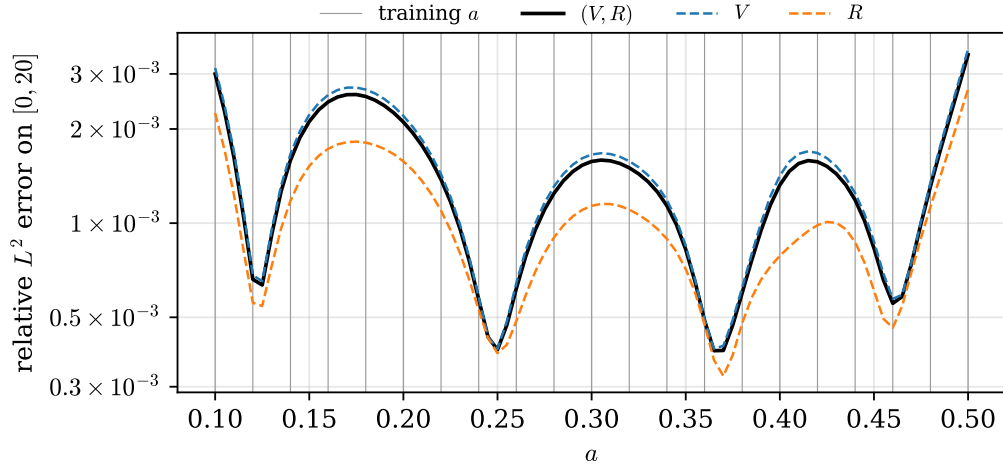


Figure 3: Relative L^2 error on $[0, 20]$ against RK45 for 81 values of a . Grey lines mark the 21 training values.

9 Limitations

- **Precision.** In float32 the loss stops decreasing at about 10^{-6} , which limits the accuracy. The same code runs in float64 on the CPU; how much lower the error gets then has not been measured.
- **Errors carry forward.** Each window is trained once and then frozen, so its error is passed on to all later windows through $y_{k+1}(a)$.

- **Range of a .** The networks are trained on $a \in [0.1, 0.5]$ only. Outside that range $\hat{a} \notin [-1, 1]$, and nothing constrains the solution there.
- **Capacity.** Each window uses the same 3×32 network as the fixed- a notebook, although it now has to cover the whole range of a . A wider network would give it more capacity; this has not been tested.

Reference

J. O. Ramsay, G. Hooker, D. Campbell and J. Cao (2007). Parameter estimation for differential equations: a generalized smoothing approach. *Journal of the Royal Statistical Society: Series B*, 69(5), 741–796.